LOAN APPROVAL PREDICTION
USING THE ID3 DECISION TREE

DWDM - CSA1614

Name: Rohith Manikanta Rellu
Register Number: 192425326

Aim

To create a sample loan application dataset and use the ID3 decision tree algorithm to predict whether a loan application should be approved based on income, credit score, and existing loan status.

Software and Libraries Required

Python in Jupyter Notebook, Pandas, NumPy, Matplotlib, and Python's built-in collections module.

Tasks Performed

1. Create a sample dataset containing at least 10 records.
2. Identify the target class and predictor attributes.
3. Construct a decision tree using the ID3 algorithm.
4. Show the attribute selection process using Information Gain for at least one split.
5. Draw the final decision tree and classify two new applicants.

Task 1: Create a Sample Dataset

A sample dataset containing 12 loan applicants is created. Each record contains three predictor attributes—Income, Credit Score, and Existing Loan—and one target attribute, Loan Approval. The data is artificial and is used only to demonstrate the algorithm.

Applicant ID	Income	Credit Score	Existing Loan	Loan Approval
1	High	Good	No	Yes
2	High	Good	Yes	Yes
3	High	Poor	No	No
4	High	Poor	Yes	No
5	Medium	Good	No	Yes
6	Medium	Good	Yes	Yes
7	Medium	Poor	No	No
8	Medium	Poor	Yes	No
9	Low	Good	No	Yes
10	Low	Good	Yes	No
11	Low	Poor	No	No
12	Low	Poor	Yes	No

Jupyter Notebook Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from collections import Counter

data = {
    'Income': ['High', 'High', 'High', 'High',
               'Medium', 'Medium', 'Medium', 'Medium',
               'Low', 'Low', 'Low', 'Low'],
    'Credit_Score': ['Good', 'Good', 'Poor', 'Poor',
                     'Good', 'Good', 'Poor', 'Poor',
                     'Good', 'Good', 'Poor', 'Poor'],
    'Existing_Loan': ['No', 'Yes', 'No', 'Yes',
                      'No', 'Yes', 'No', 'Yes',
                      'No', 'Yes', 'No', 'Yes'],
    'Loan_Approval': ['Yes', 'Yes', 'No', 'No',
                      'Yes', 'Yes', 'No', 'No',
                      'Yes', 'No', 'No', 'No']
}

df = pd.DataFrame(data)
df.index = df.index + 1
df.index.name = 'Applicant_ID'
df
```

Jupyter Output 1: Sample Dataset

Applicant_ID	Income	Credit_Score	Existing_Loan	Loan_Approval
1	High	Good	No	Yes
2	High	Good	Yes	Yes
3	High	Poor	No	No
4	High	Poor	Yes	No
5	Medium	Good	No	Yes
6	Medium	Good	Yes	Yes
7	Medium	Poor	No	No
8	Medium	Poor	Yes	No
9	Low	Good	No	Yes
10	Low	Good	Yes	No
11	Low	Poor	No	No
12	Low	Poor	Yes	No

Task 2: Identify the Target Class and Predictor Attributes

The target class is the attribute that the model is expected to predict. The predictor attributes are the input features used by the algorithm to make that prediction.

Attribute	Role	Possible Values / Meaning
Income	Predictor	High, Medium, Low
Credit_Score	Predictor	Good, Poor
Existing_Loan	Predictor	Yes, No
Loan_Approval	Target class	Yes (Approved), No (Not Approved)

Therefore, Loan_Approval is the target class, while Income, Credit_Score, and Existing_Loan are the predictor attributes.

Task 3: Construct a Decision Tree Using the ID3 Algorithm

ID3 (Iterative Dichotomiser 3) builds a decision tree by selecting the attribute with the highest Information Gain at each split. The process continues recursively until the records in a branch belong to one class or no predictor attributes remain.

Step 1: Calculate Entropy

Entropy measures the uncertainty in the target class. It is calculated using the formula:

$$H(S) = - \Sigma\ p_i \log_2(p_i)$$

In the sample dataset, 5 applicants are approved and 7 are not approved. Therefore:

$$H(S) = -(5/12 \times \log_2(5/12)) - (7/12 \times \log_2(7/12)) \approx 0.9799$$

```
def entropy(labels):
    counts = Counter(labels)
    total = len(labels)
    result = 0

    for count in counts.values():
        p = count / total
        result -= p * np.log2(p)

    return result

target_entropy = entropy(df['Loan_Approval'])
print("Total records:", len(df))
print("Approved:", (df['Loan_Approval'] == 'Yes').sum())
print("Not Approved:", (df['Loan_Approval'] == 'No').sum())
print("Entropy of dataset:", round(target_entropy, 4))
```

Jupyter Output 3: Entropy Calculation

```
Total records: 12
Approved: 5
Not Approved: 7
Entropy of dataset: 0.9799
```

Step 2: Calculate Information Gain

Information Gain measures how much uncertainty is reduced when the dataset is split using a particular attribute. The attribute with the highest Information Gain is selected.

$$IG(S, A) = H(S) - \Sigma\ (|S_v| / |S|)\ H(S_v)$$

For Credit_Score, the Good group contains 6 applicants: 5 approved and 1 not approved. The Poor group contains 6 applicants, all not approved.

Entropy(Good) = $-(5/6 \times \log_2(5/6)) - (1/6 \times \log_2(1/6)) \approx 0.6500$

Entropy(Poor) = 0, because all six applicants are in the No class.

IG(Credit_Score) = $0.9799 - [(6/12 \times 0.6500) + (6/12 \times 0)] \approx 0.6549$

```
def information_gain(data, attribute, target):
    total_entropy = entropy(data[target])
    weighted_entropy = 0

    for value in data[attribute].unique():
        subset = data[data[attribute] == value]
        weight = len(subset) / len(data)
        weighted_entropy += weight * entropy(subset[target])

    return total_entropy - weighted_entropy


attributes = ['Income', 'Credit_Score', 'Existing_Loan']

for attribute in attributes:
    gain = information_gain(df, attribute, 'Loan_Approval')
    print(attribute, "Information Gain =", round(gain, 4))
```

Attribute	Approximate Information Gain
Income	0.0428
Credit_Score	0.6549
Existing_Loan	0.0207

Credit_Score has the highest Information Gain (approximately 0.6549), so ID3 selects it as the root node of the decision tree.

Jupyter Output 4: Information Gain for Each Attribute

```
Income Information Gain = 0.0428
Credit_Score Information Gain = 0.6549
Existing_Loan Information Gain = 0.0207
```

Task 3 (Continued): ID3 Algorithm Implementation

The following function recursively selects the attribute with the highest Information Gain. If all examples in a subset have the same target value, that class is returned as a leaf node.

```
def id3(data, attributes, target):
    labels = data[target]

    if labels.nunique() == 1:
        return labels.iloc[0]

    if len(attributes) == 0:
        return labels.mode()[0]

    best_attribute = max(
        attributes,
        key=lambda attr: information_gain(data, attr, target)
    )

    tree = {best_attribute: {}}
    remaining_attributes = [
        attr for attr in attributes if attr != best_attribute
    ]

    for value in data[best_attribute].unique():
        subset = data[data[best_attribute] == value]
        tree[best_attribute][value] = id3(
            subset, remaining_attributes, target
        )

    return tree


predictors = ['Income', 'Credit_Score', 'Existing_Loan']
decision_tree = id3(df, predictors, 'Loan_Approval')
print(decision_tree)
```

Print the Decision Tree

```
def print_tree(tree, indent=""):
    if not isinstance(tree, dict):
        print(indent + "Loan Approval: " + tree)
        return

    attribute = next(iter(tree))
    print(indent + attribute)

    for value, subtree in tree[attribute].items():
        print(indent + "  " + value + " ->")
        print_tree(subtree, indent + "    ")

print_tree(decision_tree)
```

Jupyter Output 5: Constructed ID3 Decision Tree

```
Decision Tree:
{'Credit_Score': {'Good': {'Income': {'High': 'Yes', 'Medium': 'Yes', 'Low': {'Existing_Loan': {'No': 'Yes', 'Yes': 'No'}}}}, 'Poor': 'No'}}
```

Task 4: Attribute Selection Using Information Gain

ID3 compares the Information Gain of all available predictor attributes and chooses the attribute with the largest value. In this dataset, Credit_Score gives the largest gain.

Attribute	Information Gain (approx.)	Selection
Income	0.0428	Not selected as root
Credit_Score	0.6549	Selected as root
Existing_Loan	0.0207	Not selected as root

The split on Credit_Score separates all applicants with a Poor credit score into the Not Approved class. For applicants with a Good credit score, the algorithm continues splitting on Income, and then Existing_Loan for the Low-income group.

Note: Information Gain values are based on the 12-record sample dataset and are rounded to four decimal places.

Jupyter Output 6: Root Attribute Selection / Information Gain

```
Credit_Score
  Good ->
    Income
      High ->
        Loan Approval: Yes
      Medium ->
        Loan Approval: Yes
      Low ->
        Existing_Loan
          No ->
            Loan Approval: Yes
          Yes ->
            Loan Approval: No
  Poor ->
    Loan Approval: No
```

Task 5: Draw the Final Decision Tree and Classify Two New Applicants

Final Decision Tree

The resulting decision rules are:

- If Credit_Score is Poor, predict Loan_Approval = No.
- If Credit_Score is Good and Income is High, predict Yes.
- If Credit_Score is Good and Income is Medium, predict Yes.
- If Credit_Score is Good and Income is Low, check Existing_Loan.
- For Low income with Good credit: Existing_Loan = No gives Yes; Existing_Loan = Yes gives No.

Use the following code in Jupyter Notebook to draw a clear diagram of the decision tree.

```
fig, ax = plt.subplots(figsize=(12, 7))
ax.set_xlim(0, 12)
ax.set_ylim(0, 8)
ax.axis('off')

def draw_node(x, y, text):
    ax.text(
        x, y, text, ha='center', va='center',
        bbox=dict(boxstyle='round,pad=0.45',
                  facecolor='white', edgecolor='black'),
        fontsize=10
    )

def draw_edge(x1, y1, x2, y2, label=''):
    ax.annotate(
        '', xy=(x2, y2 + 0.35), xytext=(x1, y1 - 0.35),
        arrowprops=dict(arrowstyle='->', color='black')
    )
    if label:
        ax.text((x1+x2)/2, (y1+y2)/2, label,
                ha='center', va='center', fontsize=9,
                bbox=dict(facecolor='white', edgecolor='none', pad=1))

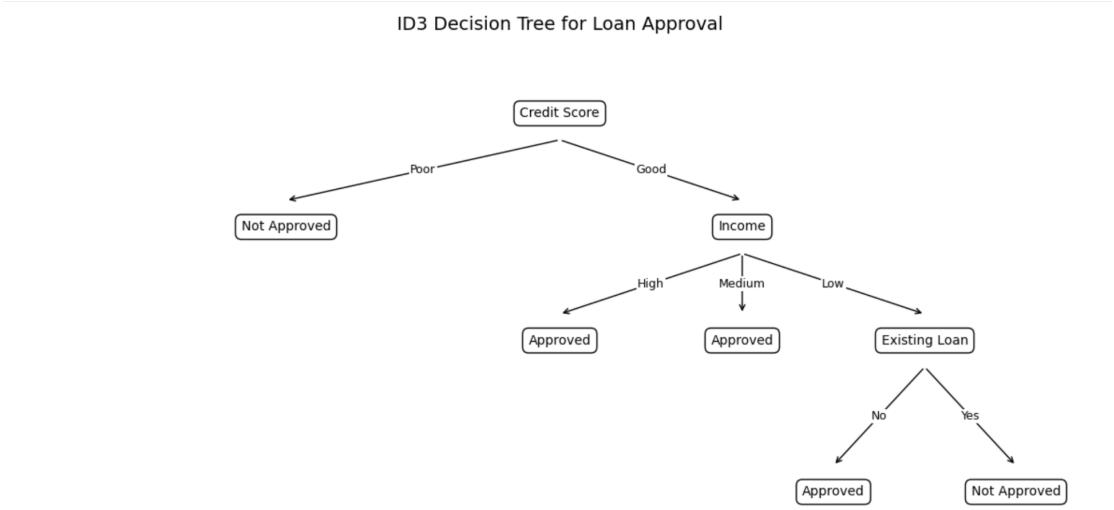
draw_node(6, 7, 'Credit Score')
draw_node(3, 5.5, 'Not Approved')
draw_node(8, 5.5, 'Income')
draw_edge(6, 7, 3, 5.5, 'Poor')
draw_edge(6, 7, 8, 5.5, 'Good')

draw_node(6, 4, 'Approved')
draw_node(8, 4, 'Approved')
draw_node(10, 4, 'Existing Loan')
draw_edge(8, 5.5, 6, 4, 'High')
draw_edge(8, 5.5, 8, 4, 'Medium')
draw_edge(8, 5.5, 10, 4, 'Low')

draw_node(9, 2, 'Approved')
draw_node(11, 2, 'Not Approved')
draw_edge(10, 4, 9, 2, 'No')
draw_edge(10, 4, 11, 2, 'Yes')

plt.title('ID3 Decision Tree for Loan Approval', fontsize=14)
plt.tight_layout()
plt.show()
```

Jupyter Output 7: Final Decision Tree Diagram



Classify Two New Applicants

Applicant	Income	Credit Score	Existing Loan	Predicted Approval
Applicant 1	Low	Good	No	Yes (Approved)
Applicant 2	High	Poor	No	No (Not Approved)

```
def predict(tree, applicant):
    if not isinstance(tree, dict):
        return tree

    attribute = next(iter(tree))
    value = applicant[attribute]

    if value not in tree[attribute]:
        return "Unknown"

    return predict(tree[attribute][value], applicant)


applicants = [
    {'Income': 'Low', 'Credit_Score': 'Good', 'Existing_Loan': 'No'},
    {'Income': 'High', 'Credit_Score': 'Poor', 'Existing_Loan': 'No'}
]
```

```
for i, applicant in enumerate(applicants, start=1):
    result = predict(decision_tree, applicant)
    print(f"Applicant {i}: {result}")
```

Jupyter Output 8: Classification of Two New Applicants

```
Applicant 1: Yes
Applicant 2: No
```

Result

The ID3 decision tree was implemented successfully using a sample dataset of 12 loan applicants. Entropy and Information Gain were calculated, and Credit_Score was selected as the root attribute because it had the highest Information Gain. The final decision tree was drawn, and two new applicants were classified.

Conclusion

This project demonstrates how the ID3 algorithm selects attributes using Information Gain and builds a decision tree for classification. Applicant 1 was predicted as Approved, while Applicant 2 was predicted as Not Approved. The dataset is artificial and the model is intended for educational demonstration only; it should not be used to make real lending decisions.